

NoSQL Databases

1. Overview

NoSQL means **Not Only SQL** — databases that do **not strictly follow the traditional relational SQL model**.

They typically relax one or more SQL characteristics such as:

- Fixed schema
- Table relationships
- Joins
- Strong ACID transactions across all operations
- Vertical scaling dependence

NoSQL systems are often designed for:

- Horizontal scalability
- High throughput
- Flexible schema
- Large distributed systems
- Low latency at scale

2. Important Clarification

NoSQL does **not** mean “bad” or “no structure.”

It means:

Alternative database models optimized for modern scale and distributed workloads.

Many NoSQL databases support some transactions and query languages.

3. Common Types of NoSQL Databases

Type	Description	Example Use
Key-Value	key → value storage	Sessions, cache
Document	JSON-like records	User profiles, CMS
Wide Column	Sparse distributed tables	Analytics, logs
Graph	Nodes + edges	Social networks

4. Popular Examples

- MongoDB
- Apache Cassandra
- Redis
- Amazon DynamoDB
- Neo4j

5. Why NoSQL Became Popular

Traditional SQL systems may face challenges with:

- Massive internet-scale traffic
- Rapid schema changes
- Global distribution
- Huge write volume
- Unstructured data

NoSQL systems were designed to address these needs.

6. Key Characteristics

A. Flexible Schema

Different records may have different fields.

Example:

JSON

```
{  
  "user_id": 101,  
  "name": "Samarth",  
  "skills": ["React", "Node"]  
}
```

Another record may include extra fields.

B. Horizontal Scaling

Scale by adding machines.

None

Node1 + Node2 + Node3

Useful for large traffic systems.

C. High Availability

Often replicate across nodes/regions.

D. Eventual Consistency (Common)

Some NoSQL systems prioritize availability over immediate consistency.

7. NoSQL vs SQL

Feature	SQL	NoSQL
Schema	Fixed	Flexible
Joins	Strong	Limited / avoided
Scaling	Vertical + some horizontal	Horizontal first
Transactions	Strong ACID	Varies
Consistency	Strong often	Tunable / eventual often

Best for	Structured business data	Large-scale distributed data
----------	--------------------------	------------------------------

8. Type 1: Key-Value Store

Simple:

None

`session_101 → {...}`

`cart_55 → {...}`

Best for:

- Sessions
- Cache
- Shopping carts
- Fast lookups

Example:

- Redis

9. Type 2: Document Database

Stores JSON/BSON documents.

Example:

JSON

```
{  
  
  "product_id": 501,  
  
  "name": "Laptop",  
  
  "price": 65000  
  
}
```

Best for:

- Product catalogs
- User profiles
- Dynamic content

Example:

- MongoDB

10. Type 3: Wide Column Store

Stores rows with flexible columns across clusters.

Best for:

- Time-series
- Event logs
- Huge write throughput

Example:

- Apache Cassandra

11. Type 4: Graph Database

Stores relationships directly.

None

User A → follows → User B

Best for:

- Social graphs
- Recommendations
- Fraud rings

Example:

- Neo4j

12. CAP Trade-offs

Many NoSQL systems are designed around distributed trade-offs:

Choose among:

- Consistency
- Availability
- Partition tolerance

Often availability + partition tolerance are emphasized.

13. Best Use Cases

Use NoSQL when you need:

- Massive scale
- Flexible schema
- Global distribution
- Very high write throughput
- Low-latency key lookups
- Semi-structured data

14. Example Systems Using NoSQL

- Social feeds
- Chat messages
- Product catalogs
- IoT telemetry
- Real-time analytics
- Session stores

15. Common Design Pattern

Use polyglot persistence:

None

SQL for payments/orders

NoSQL for sessions/feed/catalog

Use best DB for each workload.

16. Limitations / Trade-offs

Complex joins harder

Transactions may be limited

Data duplication common

Eventual consistency complexity

Query flexibility varies

17. Interview Answer Example

Q: Why NoSQL here?

I'd use NoSQL because the system needs horizontal scalability, flexible schema, and high write throughput across multiple nodes.

18. Short Revision Notes

None

NoSQL = Not Only SQL

Types:

- Key-value
- Document
- Wide-column
- Graph

Best for:

scale, flexibility, distributed systems

19. One-Line Summary

NoSQL databases are non-relational data stores optimized for scalability, flexible data models, distributed architectures, and high-performance workloads where traditional SQL constraints may be limiting.